

Forth Integration — Current-State Audit

Generated from a code scan of the repository as of this commit. Facts only; no speculation about what should exist.

1. Edge functions touching Forth

All live in `supabase/functions/`. All eight are configured with `verify_jwt = false` in `supabase/config.toml`. None are invoked from a database trigger. None are scheduled by code in this repo (see §3). None are webhook receivers (see §4).

Function	Plain-English purpose	Forth endpoints called	Invocation
<code>forth-auth</code>	Diagnostic / shared helper. Authenticates with Forth CRM via OAuth, returns a short token preview, and caches the access token in module memory for ~9 days.	POST <code>https://api.forthhcrm.com/v1/auth/token</code>	Client (<code>testForthAuth()</code> in <code>src/lib/forthApi.ts</code> , exposed via <code>useTestForthAuth</code> hook). Also <code>import</code> -ed by other functions as a reference helper but each Forth function re-implements <code>getAccessToken()</code> locally rather than importing it.
<code>forth-create-draft</code>	Takes a CRM <code>transaction_id</code> , looks up the linked <code>client_service</code> and <code>clients.forth_crm_id</code> , then creates a payment draft in Forth Pay. Writes the	POST <code>https://api.forthpay.com/v1/drafts</code> (+ OAuth token call)	Client — <code>usePushToForth()</code> → button in <code>src/components/payments/TransactionDetailSheet.tsx</code> .

Function	Plain-English purpose	Forth endpoints called	Invocation
	returned draft id back to <code>transactions.external_id</code> , sets status to <code>pending</code> , logs to <code>forth_sync_log</code> .		
<code>forth-cancel-draft</code>	Cancels a Forth Pay draft for a given <code>transaction_id</code> . Enforces a 7-day lock window and rejects if status is already <code>cancelled</code> or <code>cleared</code> . Sets local status to <code>cancelled</code> .	POST <code>https://api.forthpay.com/v1/drafts/{external_id}/cancel</code>	Client — <code>useCancelForthDraft()</code> → <code>TransactionDetailSheet.tsx</code> .
<code>forth-update-draft</code>	Updates a Forth Pay draft's <code>process_date</code> and/or <code>amount</code> . Same 7-day lock and same status guards as cancel.	PUT <code>https://api.forthpay.com/v1/drafts/{external_id}</code> (URL constructed in body of function; method PUT)	Client — <code>useUpdateForthDraft()</code> . Wired in <code>src/hooks/useForthApi.ts</code> ; no UI button in <code>TransactionDetailSheet</code> yet.
<code>forth-poll-transactions</code>	Pulls all local <code>transactions</code> with <code>status='pending'</code> and a non-null <code>external_id</code> (cap 100), asks Forth Pay for their current status in a batched report call, and maps Forth statuses (<code>open/pending/scheduled/processing</code>)	POST <code>https://api.forthpay.com/v1/reports/transactions</code> (body { <code>draft_ids: [...]</code> })	Client — <code>usePollForthTransactions()</code> triggers from a button in <code>TransactionDetailSheet.tsx</code> . Not scheduled anywhere in this repo.

Function	Plain-English purpose	Forth endpoints called	Invocation
	g → pending; cleared/complete d/settled → cleared; cancelled/cancel ed → cancelled; nsf/returned/reje cted/failed → failed). Writes processed_at, error_message, sync_error.		
forth-sync-client	Multi-mode client sync. Modes: <code>fetch</code> (GET contact details), <code>link</code> (write a known <code>forth_crm_id</code> onto <code>clients.forth_crm_id</code>), <code>post_note</code> (push a system note to Forth CRM).	GET <code>https://api.forthcrm.com/v1/contacts/{forth_crm_id}</code> and POST <code>https://api.forthcrm.com/v1/contacts/{forth_crm_id}/notes</code>	Client — <code>useSyncForthClient</code>, <code>useLinkClientToForth</code>, <code>usePostForthNote</code>.
forth-pause-resume	Pauses or resumes a client's drafts in Forth CRM.	POST <code>https://api.forthcrm.com/v1/contacts/{forth_crm_id}/pause</code> and <code>.../resume</code>	Client — <code>usePauseForthClient</code> / <code>useResumeForthClient</code>. Not wired into the UI yet — hooks exist, no component currently calls them.
forth-register-client	One-shot enrollment registration: creates contact, attaches bank account, attaches debts, and enrolls in Forth Pay. Writes returned id to <code>clients.forth_crm_id</code>	POST <code>https://api.forthcrm.com/v1/clients</code>, POST <code>.../bank-accounts</code>, POST <code>.../debts</code>, POST	Client (background) — called from <code>src/components/enrollment/EnrollmentWizard.tsx</code> (step 8, fire-and-forget, wrapped in try/catch

Function	Plain-English purpose	Forth endpoints called	Invocation
	<code>m_id</code> . Bank and debt failures are logged but non-blocking; only contact-create failure aborts.	<code>.../clients/{id}/enroll</code>	with "non-blocking" comment), and from <code>src/components/services/ServiceDetailSheet.tsx</code> "Register with Forth Pay" button via <code>useRegisterForthClient</code> .

Two hosts are in use:

- **Forth CRM:** <https://api.forthcrm.com/v1/...> (auth, contacts, notes, clients, bank-accounts, debts, enroll, pause/resume)
- **Forth Pay:** <https://api.forthpay.com/v1/...> (drafts, reports/transactions)

2. Database functions and triggers

Result: none.

A `grep` of all `db-functions` in the project (24 functions) shows zero references to `forth`, `forth_crm_id`, `forth_sync_log`, or `external_id`. The DB does not read from or write to any Forth-sourced column. The `db-triggers` listing for this project is empty.

All Forth side effects happen in edge functions invoked from the client; the database is a passive store.

3. Scheduled jobs

Result: none in this repo.

- No `cron.schedule(...)` call exists in `supabase/migrations/` or anywhere else in the codebase.
- No `pg_cron` extension setup migration touches Forth.

- The `forth-poll-transactions` function is structured to be polled (it caps itself at 100 transactions and is idempotent), but it is only invoked from a manual UI button (`usePollForthTransactions` in `TransactionDetailSheet.tsx`). A live `cron.job` table query was blocked by RLS (`permission denied for schema cron`), so any schedule added directly in the Supabase dashboard outside source control would not show here — worth confirming in-dashboard before extending.
-

4. Inbound webhooks

Result: none.

- No `forth-webhook` function exists under `supabase/functions/`.
 - `supabase/config.toml` lists `docuseal-webhook` but no Forth equivalent.
 - All status changes from Forth into the local DB happen via outbound polling (`forth-poll-transactions`), not push.
-

5. Forth credentials

Two project-level Supabase secrets, both consumed only by edge functions via `Deno.env.get(...)`:

Secret	Used as	Notes
<code>FORTH_CLIENT_ID</code>	OAuth <code>client_id</code> in the POST to <code>https://api.forthcrm.com/v1/auth/token</code>	Single value. No per-tenant / per-company variant.
<code>FORTH_API_KEY</code>	OAuth <code>client_secret</code> (passed through a <code>normalize()</code> helper that strips whitespace and newlines)	Single value. No rotation logic.

Credential pattern: a **single global Forth account**. Every edge function calls `getAccessToken()` with the same `FORTH_CLIENT_ID` / `FORTH_API_KEY`, regardless of which CRM company / `company_id` the local record belongs to.

There is a table `public.company_processor_configs` (migration `20260131231530_...`) with a per-company `api_key_encrypted TEXT` column intended for processor-specific credentials, but **no edge function reads from it** — it is currently unused for Forth auth.

The Forth access token is cached in **module-scope memory** inside each function (`let cachedToken`), expiry hard-coded to "9 days from issue" with a 5-minute refresh buffer. Cache resets on every cold start and is not shared across function instances or across the eight separate functions.

6. Forth API operations implemented

Operation	Status	Where
Account / contact — create	Working	<code>forth-register-client</code> step 1 (POST <code>/v1/clients</code>)
Account / contact — update	Not implemented — no <code>PUT /contacts/{id}</code> or <code>PUT /clients/{id}</code> call anywhere.	
Account / contact — close / delete	Not implemented.	
Account / contact — read	Working	<code>forth-sync-client</code> (GET <code>/v1/contacts/{id}</code>)
Account — pause / resume	Working	<code>forth-pause-resume</code> (POST <code>/contacts/{id}/pause / resume</code>). Hooks defined; not wired to UI.
Bank account — add	Working	<code>forth-register-client</code> step 2 (POST <code>/bank-accounts</code>). Failure is logged but non-blocking.
Bank account — update / delete / list	Not implemented.	

Operation	Status	Where
Debt — add	Working	<code>forth-register-client</code> step 3 (POST <code>/debts</code>). Failure is logged but non-blocking.
Debt — update / delete / list	Not implemented.	
Enroll client in program	Working	<code>forth-register-client</code> step 4 (POST <code>/clients/{id}/enroll</code>). Failure is logged but non-blocking.
Draft — create / schedule	Working	<code>forth-create-draft</code> (POST <code>/v1/drafts</code>)
Draft — modify	Working, with 7-day lock guard	<code>forth-update-draft</code> (PUT <code>/v1/drafts/{id}</code>)
Draft — cancel	Working, with 7-day lock guard	<code>forth-cancel-draft</code> (POST <code>/v1/drafts/{id}/cancel</code>)
Draft / transaction — status fetch (poll)	Working	<code>forth-poll-transactions</code> (POST <code>/v1/reports/transactions</code>)
Note — post to contact	Working	<code>forth-sync-client</code> action <code>post_note</code> (POST <code>/contacts/{id}/notes</code>)
Payment-to-creditor request	Not implemented. No call to any Forth disbursement / settlement-funding endpoint.	
Payment-to-creditor status	Not implemented.	
Balance fetch (escrow / SPA balance)	Not implemented. No call to a Forth balance endpoint. Local escrow projection (<code>useEscrowProjection</code>) is	

Operation	Status	Where
	computed from the local <code>transactions</code> ledger only.	
Hold — place / release	Not implemented as a distinct operation. Pause/resume is the closest existing primitive; no separate hold API is called.	

7. Client-side direct API calls to Forth

Result: none.

Every Forth call originates inside `supabase/functions/forth-*/index.ts`. The frontend touches Forth exclusively through `src/lib/forthApi.ts` → `supabase.functions.invoke('forth-...')` → edge function. Confirmed by grepping `api.forthcrm.com` and `api.forthpay.com` across `src/` — zero hits outside `src/lib/docs/` (documentation strings).

`src/lib/forthApi.ts` is the de-facto adapter. No refactor needed on this axis; the boundary already exists.

8. Known issues, TODOs, workarounds noted in code

No `TODO`, `FIXME`, `XXX`, `HACK`, or `workaround` comments exist in any of the Forth edge functions, `src/lib/forthApi.ts`, or `src/hooks/useForthApi.ts`. The following are quirks visible in code without being labeled as such:

1. **OAuth response shape is uncertain.** Every `getAccessToken()` implementation pulls the token from one of six possible paths: `response.access_token`, `response.api_key`, `access_token`, `api_key`, `data.access_token`, or `token`. The token is also passed back to Forth in an `Api-Key:` header rather than `Authorization: Bearer`. The code comments hint at past response-shape instability.

2. **API secret has to be normalized.** A `normalize()` helper strips whitespace and newlines from `FORTH_API_KEY` before sending — implies the secret was at some point pasted with a trailing newline.
3. **Token cache is per-function, in-memory, with a hard-coded 9-day TTL.** Eight functions each maintain their own cache, so token refresh happens up to eight times per cold-start cycle. There is no shared cache and no server-side refresh mechanism.
4. **`getAccessToken()` is duplicated.** `forth-auth/index.ts` exports the helper, but the other seven functions each redefine their own copy rather than importing it. Any auth fix must be made in eight places.
5. **7-day lock window is enforced client-side / function-side, not by Forth response.** Both `forth-cancel-draft` and `forth-update-draft` reject locally if `scheduled_date - now <= 7 days`, with the message "Contact Forth Pay directly." The local `canModifyDraft()` helper in `src/lib/forthApi.ts` mirrors this for UI gating.
6. **Forth status responses are loosely typed.** `forth-poll-transactions` extracts the transaction list from any of `result.response.transactions`, `result.transactions`, or `[]`, and matches by either `t.id` or `t.draft_id`. NSF is detected by string match on `error_message` containing "insufficient" OR `return_code === 'R01'` OR `status === 'nsf'`.
7. **Bank, debt, and enroll failures inside `forth-register-client` are non-blocking.** They are logged to `forth_sync_log` with `success=false` but the function still returns `success: true` and writes `forth_crm_id` to the client. The enrollment wizard then continues even if the entire Forth registration throws (see `EnrollmentWizard.tsx` line 296, comment: "non-blocking").
8. **Sensitive fields are masked only in `forth_sync_log`, not at the wire.** `forth-register-client` masks `account_number` / `routing_number` / `ssn` in the log payload but the full values are still sent over the wire to Forth (which is expected).
9. **No rate-limit handling.** No 429 detection, no backoff, no retry queue. `forth-poll-transactions` issues one HTTP call per pending transaction sequentially with no delay.

9. Source-of-truth / linkage model

Two foreign keys, both stored as `TEXT` on local rows. There is no copy of Forth objects in the local DB and no webhook-synced shadow table — every Forth-side read is a just-in-time call.

Local row	Local column	Points to	Type	Set by	Used by
<code>clients.forth_crm_id</code>	UNIQUE, indexed (<code>idx_clients_forth_crm_id</code>), nullable	The Forth CRM contact id (10-digit numeric, stored stringified)	TEXT	Written by <code>forth-register-client</code> after successful contact create, or by <code>forth-sync-client</code> action link	Read by every other Forth function to identify the contact
<code>transactions.external_id</code>	indexed (existing <code>idx_transactions_*</code>), nullable	The Forth Pay draft id (numeric, stored stringified)	TEXT	Written by <code>forth-create-draft</code> on successful draft creation	Read by <code>forth-cancel-draft</code> , <code>forth-update-draft</code> , <code>forth-poll-transactions</code>

Plus two telemetry columns added in migration `20260203181426_...`:

- `transactions.last_sync_at` TIMESTAMPTZ — wall-clock of the most recent Forth interaction for this transaction.
- `transactions.sync_error` TEXT — last error payload (raw JSON string) from a failed Forth call.

And one audit table:

- `public.forth_sync_log` (migration `20260203181426_...`) — append-only audit. Columns: `entity_type` (`transaction` | `client` | `draft`), `entity_id` UUID, `action` (`create` | `update` | `poll` | `cancel` | `pause` | `resume` | `sync`), `request_payload` JSONB, `response_payload` JSONB, `success` BOOLEAN, `error_message` TEXT, `created_at`. RLS: authenticated staff can `SELECT`, only service role can `INSERT` (via edge functions). Indexed on (`entity_type`, `entity_id`) and (`created_at` DESC).

Linkage pattern, summarized:

- Contacts and drafts: **foreign key on external_id / forth_crm_id**, set once, never re-validated.
- Status, balances, escrow projection, payment history: **just-in-time read** (via **forth-poll-transactions** or **forth-sync-client fetch**); not mirrored locally.
- All write paths: **fire-and-acknowledge** — Forth response is logged to **forth_sync_log**; only minimal fields (status, external_id) propagate back to the canonical table.
- There is no reconciliation job and no "Forth says X but DB says Y" detector.